11.5 Hermitian Matrices

The complex analog of a real symmetric matrix is a Hermitian matrix, satisfying equation (11.0.4). Jacobi transformations can be used to find eigenvalues and eigenvectors, as can Householder reduction to tridiagonal form followed by QL iteration. Complex versions of the previous routines `jacobi`, `tred2`, and `tqli` are quite analogous to their real counterparts. For working routines, consult [1,2].

An alternative, using the routines in this book, is to convert the Hermitian problem to a real symmetric one: If $\mathbf{C} = \mathbf{A} + i\mathbf{B}$ is a Hermitian matrix, then the $n \times n$ complex eigenvalue problem

$$(\mathbf{A} + i\mathbf{B}) \cdot (\mathbf{u} + i\mathbf{v}) = \lambda(\mathbf{u} + i\mathbf{v}) \tag{11.5.1}$$

is equivalent to the $2n \times 2n$ real problem

$$\begin{bmatrix} \mathbf{A} & -\mathbf{B} \\ \mathbf{B} & \mathbf{A} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{u} \\ \mathbf{v} \end{bmatrix} = \lambda \begin{bmatrix} \mathbf{u} \\ \mathbf{v} \end{bmatrix} \tag{11.5.2}$$

Note that the $2n \times 2n$ matrix in (11.5.2) is symmetric: $\mathbf{A}^T = \mathbf{A}$ and $\mathbf{B}^T = -\mathbf{B}$ if $\mathbf{C}$ is Hermitian.

Corresponding to a given eigenvalue λ, the vector

$$\begin{bmatrix} -\mathbf{v} \\ \mathbf{u} \end{bmatrix} \tag{11.5.3}$$

is also an eigenvector, as you can verify by writing out the two matrix equations implied by (11.5.2). Thus, if $\lambda_0, \lambda_1, \ldots, \lambda_{n-1}$ are the eigenvalues of $\mathbf{C}$, then the $2n$ eigenvalues of the augmented problem (11.5.2) are $\lambda_0, \lambda_0, \lambda_1, \lambda_1, \ldots, \lambda_{n-1}, \lambda_{n-1}$; each, in other words, is repeated twice. The eigenvectors are pairs of the form $\mathbf{u} + i\mathbf{v}$ and $i(\mathbf{u} + i\mathbf{v})$; that is, they are the same up to an inessential phase. Thus we solve the augmented problem (11.5.2) and choose one eigenvalue and eigenvector from each pair. These give the eigenvalues and eigenvectors of the original matrix $\mathbf{C}$.

Working with the augmented matrix requires a factor of two more storage than the original complex matrix. In principle, a complex algorithm is also a factor of two more efficient in computer time than is the solution of the augmented problem. In practice, most complex implementations do not achieve this factor unless they are written entirely in real arithmetic. (Good library routines always do this.)

CITED REFERENCES AND FURTHER READING:

Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer).[1]

Smith, B.T., et al. 1976, *Matrix Eigensystem Routines — EISPACK Guide*, 2nd ed., vol. 6 of Lecture Notes in Computer Science (New York: Springer).[2]

11.6 Real Nonsymmetric Matrices

The algorithms for symmetric matrices given in the preceding sections are highly satisfactory in practice. By contrast, it is impossible to design equally satisfactory

algorithms for the nonsymmetric case. There are two reasons for this. First, the eigenvalues of a nonsymmetric matrix can be very sensitive to small changes in the matrix elements. Second, the matrix itself can be defective, so that there is no complete set of eigenvectors. We emphasize that these difficulties are intrinsic properties of certain nonsymmetric matrices, and no numerical procedure can "cure" them. The best we can hope for are procedures that don't exacerbate such problems.

The presence of rounding error can only make the situation worse. With finite-precision arithmetic, one cannot even design a foolproof algorithm to determine whether a given matrix is defective or not. Thus current algorithms generally *try* to find a *complete* set of eigenvectors and rely on the user to inspect the results. If any eigenvectors are almost parallel, the matrix is probably defective.

The strategy for finding the eigensystem of a general matrix parallels that of the symmetric case. First we reduce the matrix to a simpler form, and then we perform an iterative procedure on the simplified matrix. The simpler structure we use here is called *Hessenberg* form, defined later in this section. The user interface to the routine is very simple. The declaration

```
Unsymmeig h(a);
```

computes all eigenvalues and eigenvectors of the matrix `a`. The eigenvalues are stored in the complex vector `h.wri` and the corresponding eigenvectors in the columns of the matrix `h.zz`. If `h.wri[i]` is real, the real eigenvector is in `h.zz[0..n-1][i]`. For complex eigenvalues, if `h.wri[i]` has a positive imaginary part, then the complex-conjugate eigenvalue is in `h.wri[i+1]`. Only the eigenvector corresponding to `h.wri[i]` is returned, with the real part in `h.zz[0..n-1][i]` and the imaginary part in `h.zz[0..n-1][i+1]`. The eigenvector corresponding to `h.wri[i+1]` is simply the complex conjugate of this one.

Optional arguments allow you to compute only the eigenvalues, or to input a matrix already in Hessenberg form:

```
Unsymmeig h(a,false);          Only eigenvalues computed.
Unsymmeig h(a,true,true);      Both eigenvalues and eigenvectors, Hessenberg matrix.
```

Here is the routine. The implementations of the various components are discussed in the rest of this section and the next.

eigen_unsym.h

```
struct Unsymmeig {
```

Computes all eigenvalues and eigenvectors of a real nonsymmetric matrix by reduction to Hessenberg form followed by QR iteration.

```
    Int n;
    MatDoub a,zz;
    VecComplex wri;
    VecDoub scale;                  Stores scaling from balance.
    VecInt perm;                    Stores permutation from elmhes.
    Bool yesvecs,hessen;

    Unsymmeig(MatDoub_I &aa, Bool yesvec=true, Bool hessenb=false) :
        n(aa.nrows()), a(aa), zz(n,n,0.0), wri(n), scale(n,1.0), perm(n),
        yesvecs(yesvec), hessen(hessenb)
```

Computes all eigenvalues and (optionally) eigenvectors of a real nonsymmetric matrix `a[0..n-1][0..n-1]` by reduction to Hessenberg form followed by QR iteration. If `yesvecs` is input as `true` (the default), then the eigenvectors are computed. Otherwise, only the eigenvalues are computed. If `hessen` is input as `false` (the default), the matrix is first reduced to Hessenberg form. Otherwise it is assumed that the matrix is already in Hessenberg from. On output, `wri[0..n-1]` contains the eigenvalues of a sorted into descending order, while `zz[0..n-1][0..n-1]` is a matrix whose columns contain the corresponding

```
    eigenvectors. For a complex eigenvalue, only the eigenvector corresponding to the eigen-
    value with a positive imaginary part is stored, with the real part in zz[0..n-1][i] and the
    imaginary part in h.zz[0..n-1][i+1]. The eigenvectors are not normalized.
    {
        balance();
        if (!hessen) elmhes();
        if (yesvecs) {
            for (Int i=0;i<n;i++)          Initialize to unit matrix.
                zz[i][i]=1.0;
            if (!hessen) eltran();
            hqr2();
            balbak();
            sortvecs();
        } else {
            hqr();
            sort();
        }
    }
    void balance();
    void elmhes();
    void eltran();
    void hqr();
    void hqr2();
    void balbak();
    void sort();
    void sortvecs();
};
```

11.6.1 Balancing

The sensitivity of eigenvalues to rounding errors during the execution of some algorithms can be reduced by the procedure of *balancing*. The errors in the eigensystem found by a numerical procedure are generally proportional to the Euclidean norm of the matrix, that is, to the square root of the sum of the squares of the elements. The idea of balancing is to use similarity transformations to make corresponding rows and columns of the matrix have comparable norms, thus reducing the overall norm of the matrix while leaving the eigenvalues unchanged. A symmetric matrix is already balanced.

Balancing is a procedure with of order N^2 operations. Thus, the time taken by the procedure `balance`, given below, should never be significant compared to the total time required to find the eigenvalues. It is therefore recommended that you *always* balance nonsymmetric matrices. It never hurts, and it can substantially improve the accuracy of the eigenvalues computed for a badly balanced matrix.

The actual algorithm used is due to Osborne, as discussed in [1]. It consists of a sequence of similarity transformations by diagonal matrices $\mathbf{D}$. To avoid introducing rounding errors during the balancing process, the elements of $\mathbf{D}$ are restricted to be exact powers of the radix base employed for floating-point arithmetic (i.e., 2 for all modern machines, but 16 for some historical mainframe architectures). The output is a matrix that is balanced in the norm given by summing the absolute magnitudes of the matrix elements. This is more efficient than using the Euclidean norm, and equally effective: A large reduction in one norm implies a large reduction in the other.

Note that if the off-diagonal elements of any row or column of a matrix are all zero, then the diagonal element is an eigenvalue. If the eigenvalue happens to be ill-conditioned (sensitive to small changes in the matrix elements), it will have

relatively large errors when determined by the routine hqr (§11.7). Had we merely inspected the matrix beforehand, we could have determined the isolated eigenvalue exactly and then deleted the corresponding row and column from the matrix. You should consider whether such a pre-inspection might be useful in your application. (For symmetric matrices, the routines we gave will determine isolated eigenvalues accurately in all cases.)

The routine balance keeps track of the scale factors used in the balancing. If you are computing eigenvectors as well as eigenvalues, then the accumulated similarity transformation of the original matrix is undone by applying these scale factors in the routine balbak.

```
void Unsymmeig::balance()                                              eigen_unsym.h
```
Given a matrix a[0..n-1][0..n-1], this routine replaces it by a balanced matrix with identical eigenvalues. A symmetric matrix is already balanced and is unaffected by this procedure.

```
{
    const Doub RADIX = numeric_limits<Doub>::radix;
    Bool done=false;
    Doub sqrdx=RADIX*RADIX;
    while (!done) {
        done=true;
        for (Int i=0;i<n;i++) {                    Calculate row and column norms.
            Doub r=0.0,c=0.0;
            for (Int j=0;j<n;j++)
                if (j != i) {
                    c += abs(a[j][i]);
                    r += abs(a[i][j]);
                }
            if (c != 0.0 && r != 0.0) {            If both are nonzero,
                Doub g=r/RADIX;
                Doub f=1.0;
                Doub s=c+r;
                while (c<g) {                      find the integer power of the machine
                    f *= RADIX;                        radix that comes closest to balanc-
                    c *= sqrdx;                        ing the matrix.
                }
                g=r*RADIX;
                while (c>g) {
                    f /= RADIX;
                    c /= sqrdx;
                }
                if ((c+r)/f < 0.95*s) {
                    done=false;
                    g=1.0/f;
                    scale[i] *= f;
                    for (Int j=0;j<n;j++) a[i][j] *= g;   Apply similarity transforma-
                    for (Int j=0;j<n;j++) a[j][i] *= f;       tion.
                }
            }
        }
    }
}
```

```
void Unsymmeig::balbak()                                               eigen_unsym.h
```
Forms the eigenvectors of a real nonsymmetric matrix by back transforming those of the corresponding balanced matrix determined by balance.

```
{
    for (Int i=0;i<n;i++)
        for (Int j=0;j<n;j++)
            zz[i][j] *= scale[i];
}
```

11.6.2 Reduction to Hessenberg Form

An *upper Hessenberg* matrix has zeros everywhere below the diagonal except for the first subdiagonal row. For example, in the 6×6 case, the nonzero elements are

$$\begin{bmatrix} \times & \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times & \times \\ & \times & \times & \times & \times & \times \\ & & \times & \times & \times & \times \\ & & & \times & \times & \times \\ & & & & \times & \times \end{bmatrix}$$

By now you should be able to tell at a glance that such a structure can be achieved by a sequence of Householder transformations, each one zeroing the required elements in a column of the matrix. Householder reduction to Hessenberg form is in fact an accepted technique. An alternative, however, is a procedure analogous to Gaussian elimination with pivoting. We will use this elimination procedure since it is about a factor of two more efficient than the Householder method, and also since we want to teach you the method. It is possible to construct matrices for which the Householder reduction, being orthogonal, is stable and elimination is not, but such matrices are extremely rare in practice.

Straight Gaussian elimination is not a similarity transformation of the matrix. Accordingly, the actual elimination procedure used is slightly different. We proceed in a series of stages $r = 1, 2, \ldots, N-2$. Before the rth stage, the original matrix $\mathbf{A} \equiv \mathbf{A}_1$ has become $\mathbf{A}_r$, which is upper Hessenberg up to, but not including, row and column $r-1$. The rth stage then consists of the following sequence of operations:

- Find the element of maximum magnitude in column $r-1$ below the diagonal. If it is zero, skip the next two "bullets" and the stage is done. Otherwise, suppose the maximum element was in row r'.
- Interchange rows r' and r. This is the pivoting procedure. To make the permutation a similarity transformation, also interchange columns r' and r.
- For $i = r+1, r+2, \ldots, N-1$, compute the multiplier

$$n_{ir} \equiv \frac{a_{i,r-1}}{a_{r,r-1}}$$

  Subtract n_{ir} times row r from row i. To make the elimination a similarity transformation, also *add* n_{ir} times column i to column r.

A total of $N-2$ such stages are required.

When the magnitudes of the matrix elements vary over many orders, you should try to rearrange the matrix so that the largest elements are in the top left-hand corner. This reduces the roundoff error, since the reduction proceeds from left to right.

The routine `elmhes` keeps track of the permutations applied during the elimination. If you are computing eigenvectors, then the accumulated similarity transformation is applied to the eigenvectors by the routine `eltran`, which includes any necessary permutations. The operation count is about $5N^3/3$ for large N.

`void Unsymmeig::elmhes()` eigen_unsym.h

Reduction to Hessenberg form by the elimination method. Replaces the real nonsymmetric matrix `a[0..n-1][0..n-1]` by an upper Hessenberg matrix with identical eigenvalues. Recommended, but not required, is that this routine be preceded by `balance`. On output, the Hessenberg matrix is in elements `a[i][j]` with `i` $\leq$ `j+1`. Elements with `i` $>$ `j+1` are to be thought of as zero, but are returned with random values.

```
{
    for (Int m=1;m<n-1;m++) {           m is called r in the text.
        Doub x=0.0;
        Int i=m;
        for (Int j=m;j<n;j++) {         Find the pivot.
            if (abs(a[j][m-1]) > abs(x)) {
                x=a[j][m-1];
                i=j;
            }
        }
        perm[m]=i;                      Store permutation.
        if (i != m) {                   Interchange rows and columns.
            for (Int j=m-1;j<n;j++) SWAP(a[i][j],a[m][j]);
            for (Int j=0;j<n;j++) SWAP(a[j][i],a[j][m]);
        }
        if (x != 0.0) {                 Carry out the elimination.
            for (i=m+1;i<n;i++) {
                Doub y=a[i][m-1];
                if (y != 0.0) {
                    y /= x;
                    a[i][m-1]=y;
                    for (Int j=m;j<n;j++) a[i][j] -= y*a[m][j];
                    for (Int j=0;j<n;j++) a[j][m] += y*a[j][i];
                }
            }
        }
    }
}
```

`void Unsymmeig::eltran()` eigen_unsym.h

This routine accumulates the stabilized elementary similarity transformations used in the reduction to upper Hessenberg form by `elmhes`. The multipliers that were used in the reduction are obtained from the lower triangle (below the subdiagonal) of `a`. The transformations are permuted according to the permutations stored in `perm` by `elmhes`.

```
{
    for (Int mp=n-2;mp>0;mp--) {
        for (Int k=mp+1;k<n;k++)
            zz[k][mp]=a[k][mp-1];
        Int i=perm[mp];
        if (i != mp) {
            for (Int j=mp;j<n;j++) {
                zz[mp][j]=zz[i][j];
                zz[i][j]=0.0;
            }
            zz[i][mp]=1.0;
        }
    }
}
```

CITED REFERENCES AND FURTHER READING:

Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer).[1]

Smith, B.T., et al. 1976, *Matrix Eigensystem Routines — EISPACK Guide*, 2nd ed., vol. 6 of Lecture Notes in Computer Science (New York: Springer).[2]

Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §6.5.4.[3]